Aim:

Build a simple web application using Node.js and Express.js that supports Signup and Login, stores users in a local JSON file (users.json), shows a background image behind the form, and displays clear status messages:

- *Login successful*
- *User already exists*
- *Credentials not valid*
- *Email ID already used*

Objective:

- Implement a signup endpoint that prevents duplicate username and email.
- Implement a login endpoint that validates email + password.
- Persist user data to a JSON file using Node's fs module.
- Serve a static HTML + CSS page with a background image and two forms (Signup & Login).
- Use Fetch API to send form data to Express routes and show response messages on the page.
- Test the application with different cases and document outcomes.

Technologies Used

- Runtime: Node.js (LTS or current)
- Framework: Express.js
- Language: JavaScript (Node + Browser)
- Storage: JSON file (data/users.json)

Step-by-Step Implementation

1) Project Setup

1. Create project folder and initialize:

```
mkdir login-app && cd login-app
npm init -y
```

2. install dependencies:

```
npm install express
```

3. Create folders & files:

```
mkdir public data
echo "[]" > data/users.json
```

2) Server (Backend)

1. Create server.js and import modules (express, fs, path).
2. Add middleware:
   - express.json() and express.urlencoded({ extended: true }) for parsing.
   - express.static("public") to serve index.html and style.css.
3. Implement helpers:
   - loadUsers() → read and parse data/users.json (create if missing).
   - saveUsers(users) → write users array back to JSON.
4. Implement routes:
   - POST /signup: check duplicate username, duplicate email; if none, push { username, email, password }, save, respond with *"Signup successful! Please login."*
   - POST /login: find user by email AND password; if not found, respond *"Credentials not valid"*; else *"Login successful! Welcome "*.
5. Start server with app.listen(3000) and confirm in console.

3) Frontend (Public)

1. In public/index.html, build two forms: Signup (username, email, password) and Login (email, password).
2. Add a <script> that intercepts form submit, gathers values, and calls:
   - fetch('/signup', { method: 'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify(data) })
   - fetch('/login', { method: 'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify(data) })
3. Show responses in a message area (e.g., ...).

4. In public/style.css, set a full-screen background image and style the container (semi-transparent card, padding, rounded corners, hover effect on buttons).

4) Run & Test

1. Start server:

```
node server.js
```

Open http://localhost:3000 in a browser.

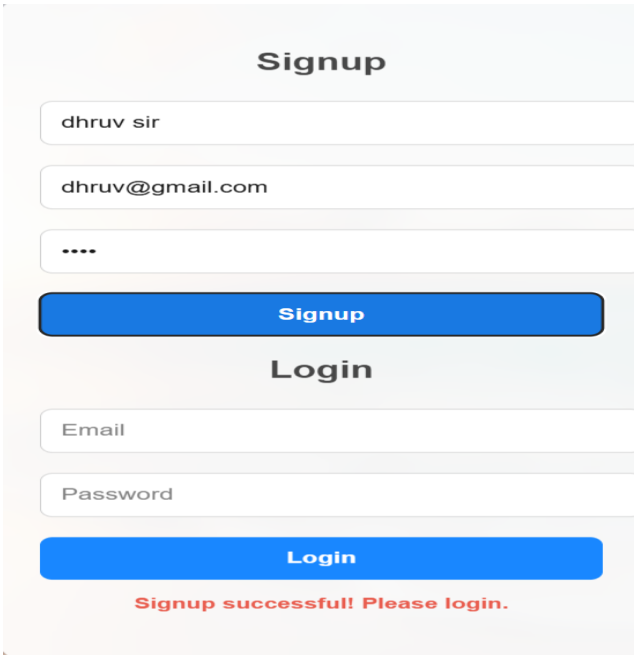
5) code of Server.js:

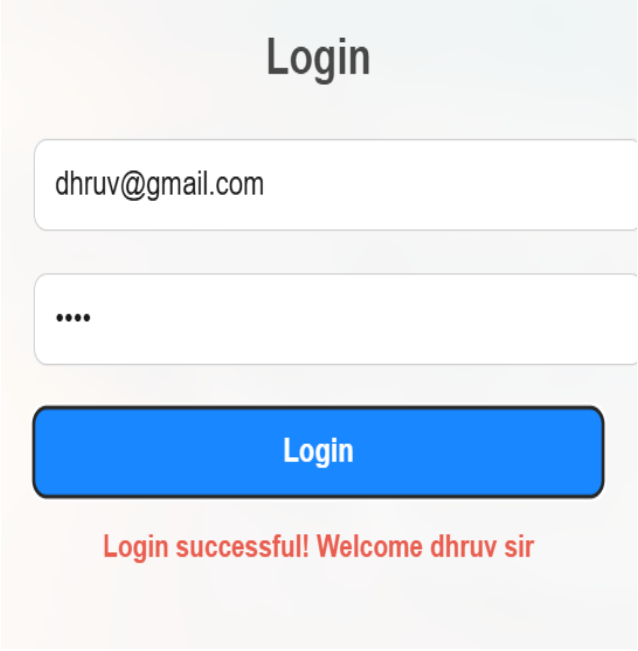
```
const express = require("express");
const fs = require("fs");
const path = require("path");

const app = express();
const PORT = 3000;

// File path for storing users
const usersFile = path.join(__dirname, "data", "users.json");

// Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static("public"));

// Load users from JSON
function loadUsers() {
  if (!fs.existsSync(usersFile)) {
    fs.writeFileSync(usersFile, JSON.stringify([]));
  }
  const data = fs.readFileSync(usersFile);
  return JSON.parse(data);
}
```

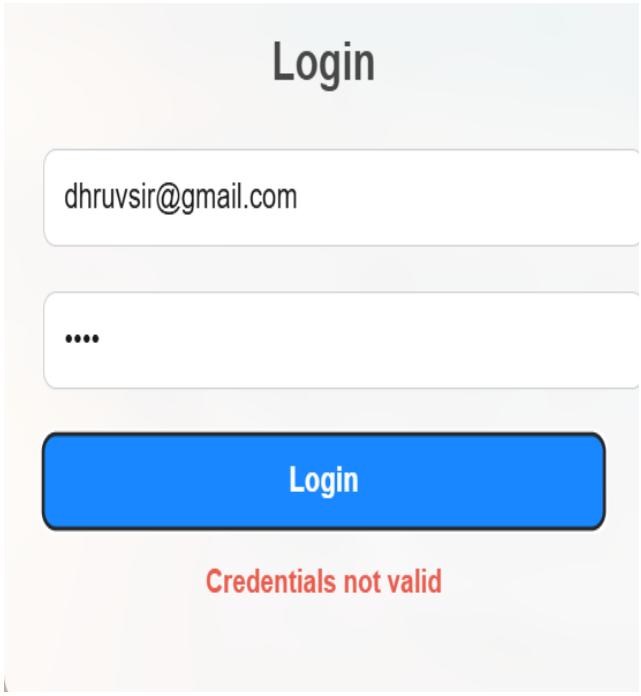
```
// Save users to JSON
function saveUsers(users) {
    fs.writeFileSync(usersFile, JSON.stringify(users, null, 2));
}

// Signup route
app.post("/signup", (req, res) => {
    const { username, email, password } = req.body;
    let users = loadUsers();

    // Check if username exists
    if (users.find(u => u.username === username)) {
        return res.json({ message: "User already exists" });
    }

    // Check if email exists
    if (users.find(u => u.email === email)) {
        return res.json({ message: "Email ID already used" });
    }

    // Save new user
    users.push({ username, email, password });
    saveUsers(users);

    return res.json({ message: "Signup successful! Please login." });
});

// Login route
app.post("/login", (req, res) => {
    const { email, password } = req.body;
    let users = loadUsers();
```

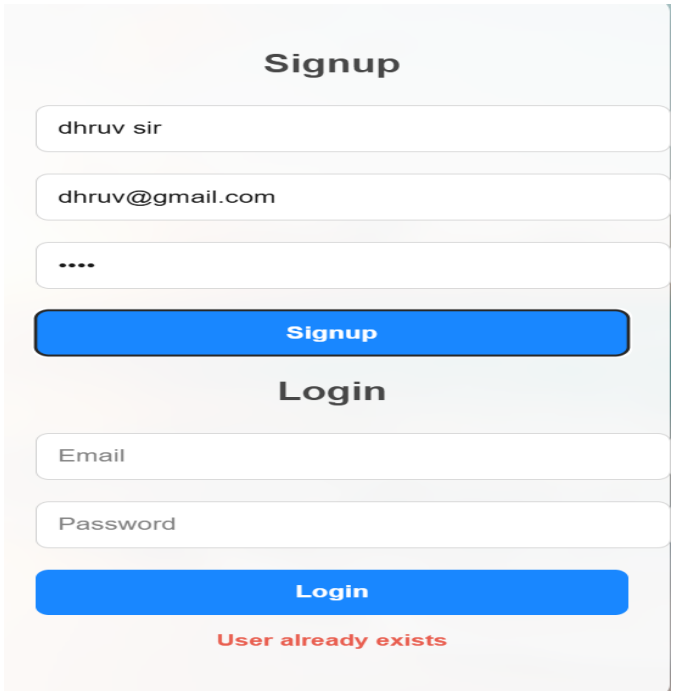
```
    const user = users.find(u => u.email === email && u.password === password);
    if (!user) {
        return res.json({ message: "Credentials not valid" });
    }

    return res.json({ message: `Login successful! Welcome ${user.username}` });
});

// Start server
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

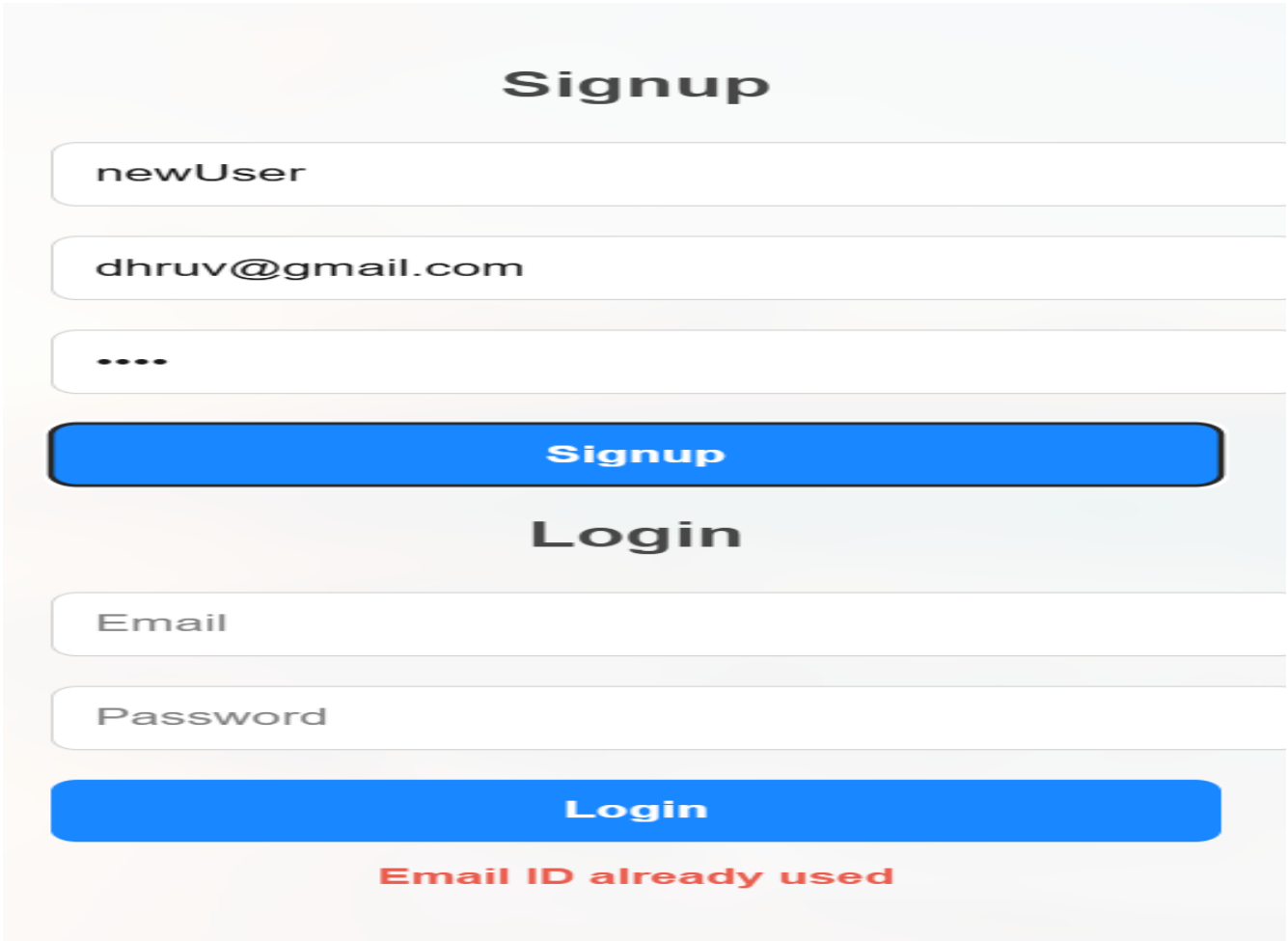
Output:

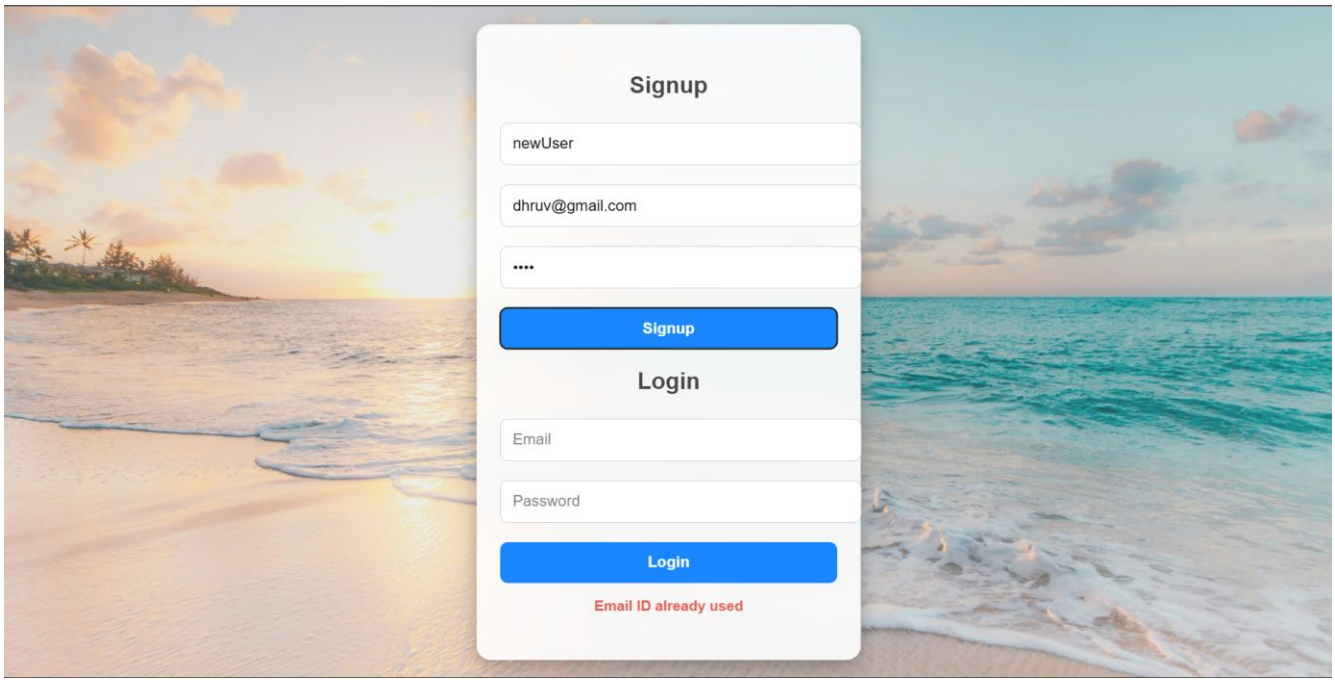





Login
dhruvsir@gmail.com
••••
Login
Credentials not valid


Signup
dhruv sir
dhruv@gmail.com
••••
Signup
Login
Email
Password
Login
User already exists


Signup
newUser
dhruv@gmail.com
••••
Signup
Login
Email
Password
Login
Email ID already used



Learning Outcomes:

- Gained hands-on experience in reading from and writing to JSON files for persistent data storage without a database.
- Understood how JSON is used to structure data and how to handle incoming client requests with middleware like express.json().
- Developed the ability to implement authentication logic with clear validation messages for different scenarios (duplicate username/email, invalid credentials).